

How Does Google Docs Work 🔥

#84: Break Into Google Docs Architecture (12 Minutes)



NEO KIM

AUG 22, 2025  PAID

Unlock access to every deep dive article by becoming a paid subscriber:

✓ Subscribed

I spent hours studying how Google Docs works so you don't have to. And I wrote this newsletter to make the key concepts simple and easy for you.

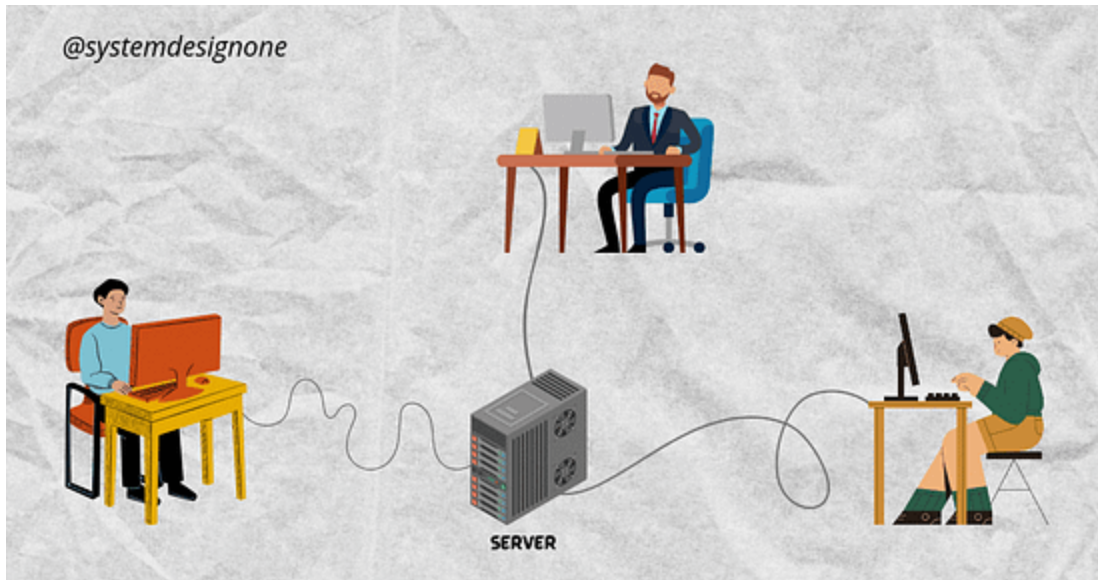
- [Share this post](#) & I'll send you some rewards for the referrals.

Note: This post is based on my research and may differ from real-world implementation.

Once upon a time, there lived a data analyst named Maria.

She emailed draft copies many times to different people to prepare monthly reports.

So she wasted a ton of time and was frustrated.



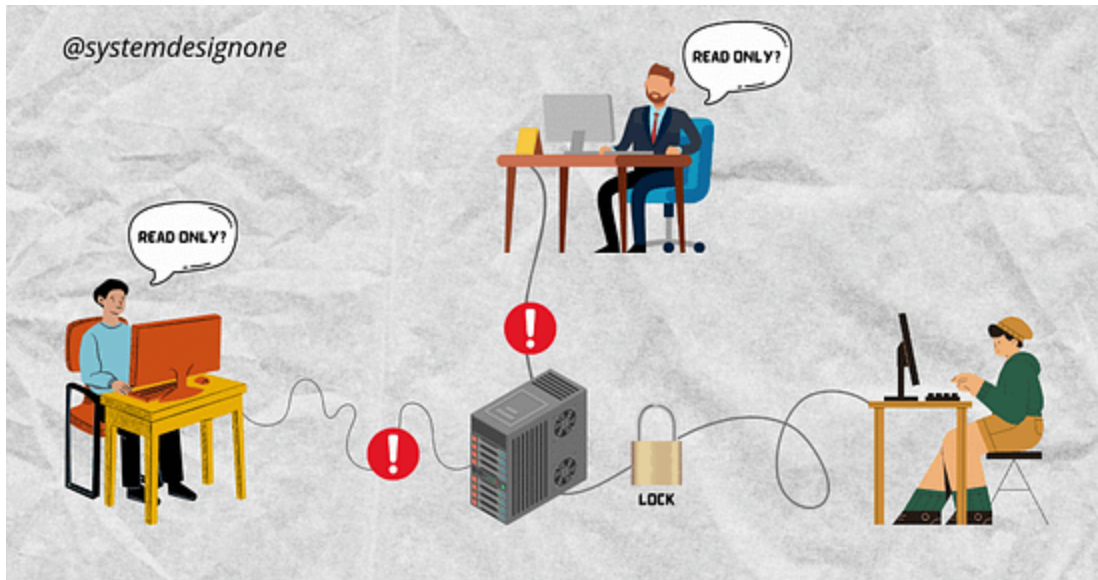
Until one day, when she decides to use Google Docs for it.

Google Docs allows collaborative editing over the internet. It means many users can work on the same document in real-time.

Yet it's difficult to implement Google Docs correctly for 3 reasons:

- Concurrent changes to the same document should converge to the same version.
- Concurrent changes to the same document must avoid conflicts.
- Any changes should be visible in real-time to each user.

Also a user should be able to make changes while they're offline.

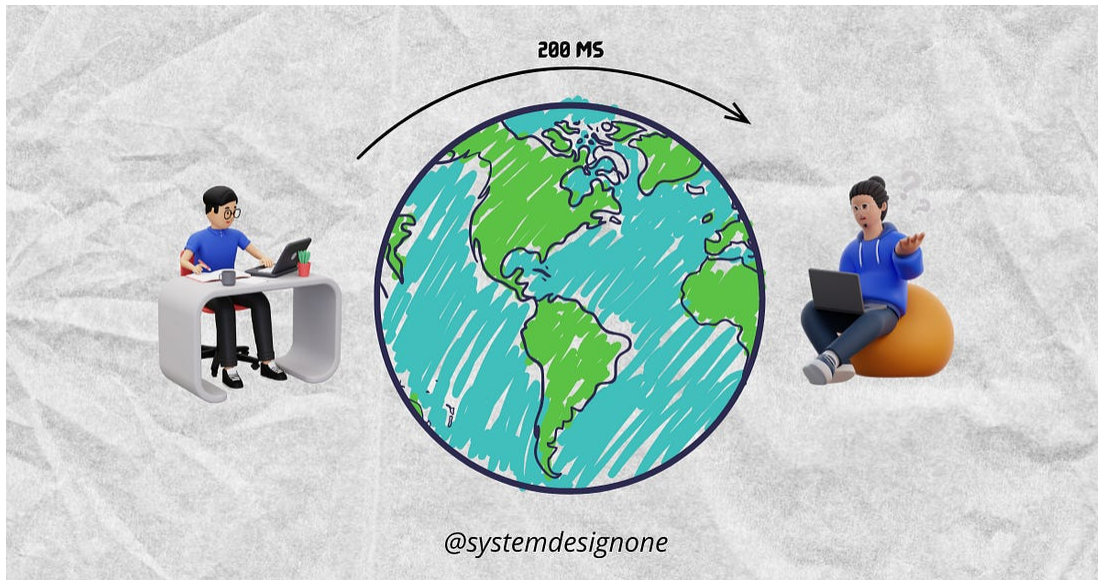


Handling Concurrency Using a Lock

A simple approach to handle concurrency is using pessimistic concurrency control.

Pessimistic locking is a mechanism for handling concurrency using a lock. It offers strong consistency, but doesn't support collaborative editing in real-time. Because it needs a central coordinator to handle data changes, only 1 user can edit at a time. Put simply, only a single document copy is available for write operations at once, while other document copies are read-only.

Besides it doesn't support offline changes.

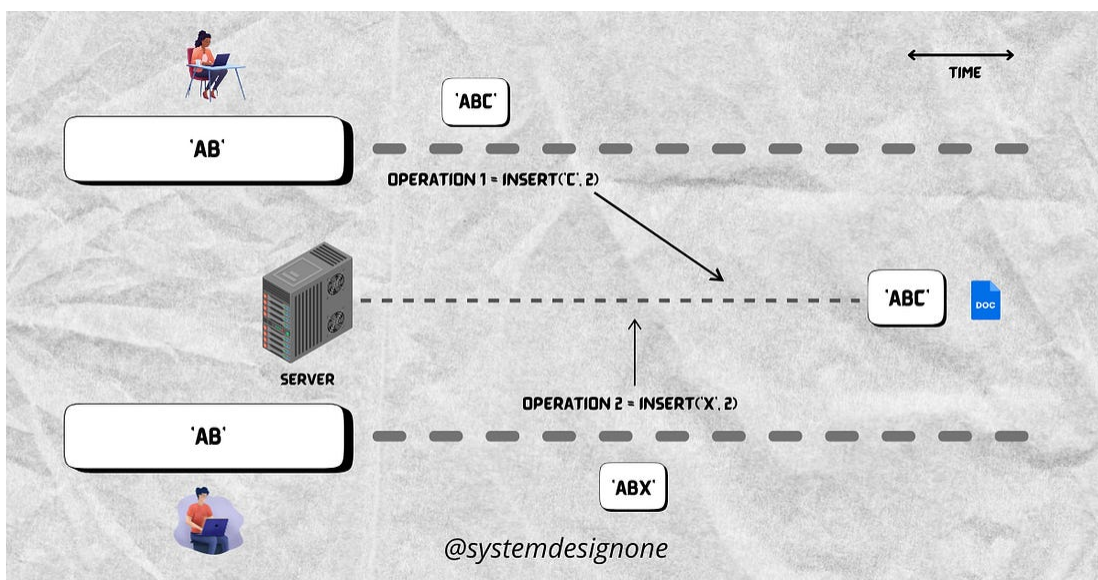


A Network Request Traveling Across Earth

Also a network round-trip across the Earth takes 200 milliseconds.

This might cause a poor user experience. So they do **latency hiding**. The idea is to keep a document copy for each user locally and then run operations locally for high responsiveness. Thus creating the illusion of lower latency than reality.

And the system propagates the changes to all users for consistency.



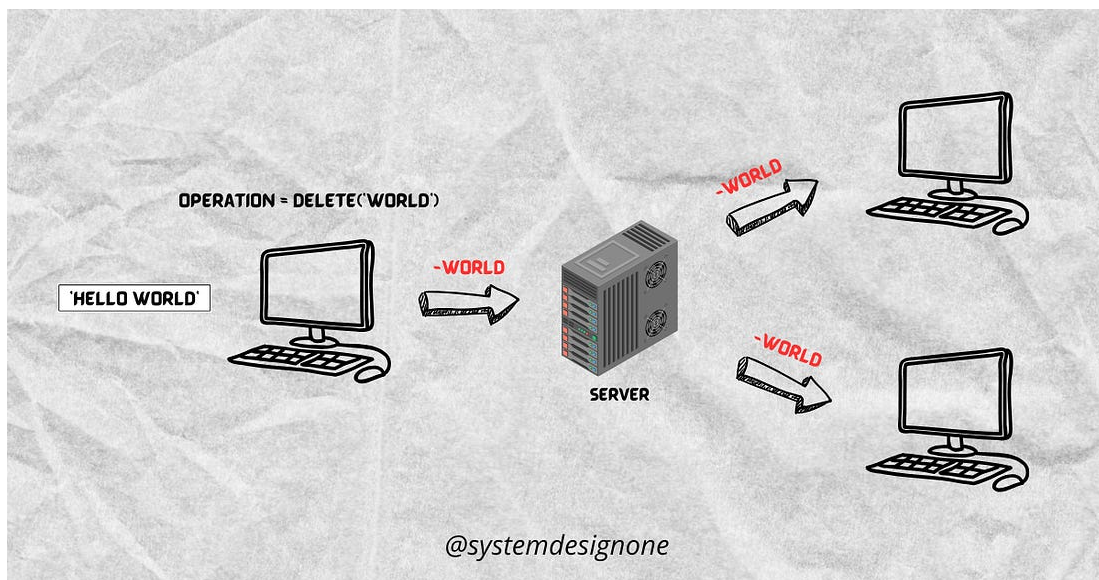
Last Write Wins Mechanism Overwriting Latest Data

A simple approach for latency hiding is using the **last-write-wins** mechanism.

Yet it resolves a conflict without waiting for coordination by applying the most recent update. So there's a risk of data loss when there are concurrent changes in high-latency networks.

It might be a good choice when concurrency is low. But it isn't suitable for this use case.

Onward.



How Differential Synchronization Works

An alternative approach to latency hiding is through **differential synchronization**.

It keeps a document copy for each user and tracks the changes locally. The system doesn't send the entire document when something changes, but only the difference (**diff**).

Yet there's a performance overhead in sending a diff for every change. Also differential synchronization only tracks diffs, and not the reason behind a change. So conflict resolution might be difficult.

While resolving conflicts manually affects the user experience.

INDEX	0	1	2	3
INSERT(TEXT = 'A', POSITION = 0)	A			
INSERT(TEXT = 'BCD', POSITION = 1)	A	B	C	D
DELETE(POSITION = 2)	A	B	D	

@systemdesignone

Operational Transformation Representing a Document Using
Revision Log

So Google Docs uses an optimistic concurrency control technique called operational transformation (OT).

OT is an algorithm to show document changes without wait times on high-latency networks. It allows different document copies to accept write operations at once. Also it handles conflict resolution automatically without locks or user interventions.

Besides OT tolerates divergence among document copies and converges them later.

Think of **operational transformation** as an event-passing mechanism; it ensures each user has the same document state even with unsynchronized changes.

With OT, the system saves each change as an event. Put simply, a change doesn't affect the underlying character of a document; instead, it adds an event to the revision log. The system then displays the document by replaying the revision log from its start.

Operational transformation saves a document as a set of operations, but it's complex to implement properly.

How Does Google Docs Work

Google Docs uses a client-server architecture for simplicity.

Here's how it works:

1. Offline Changes

They provide **read-your-write consistency** by tracking changes on the client side.

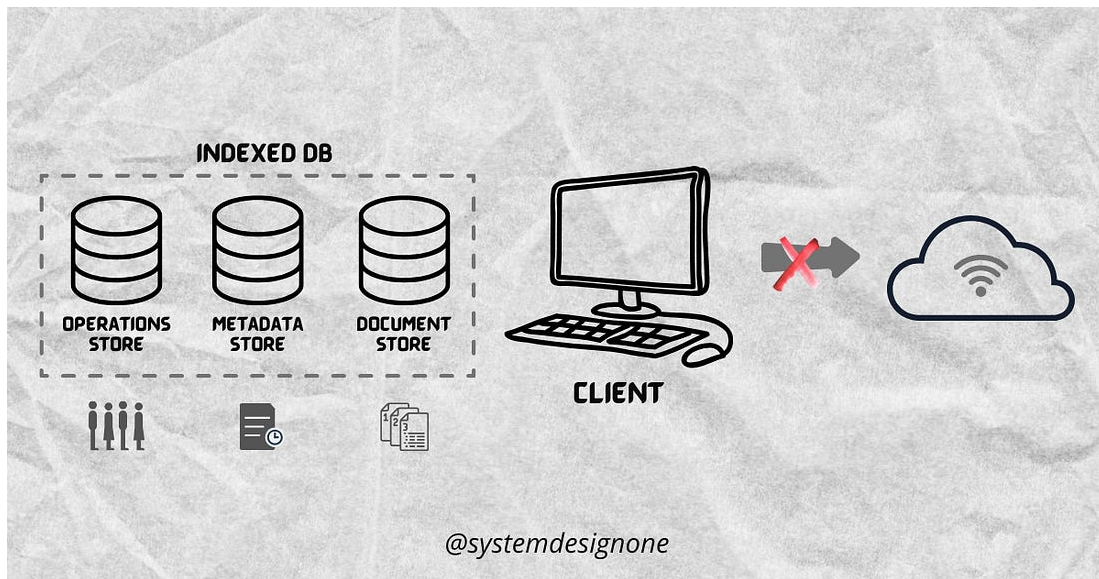
It means showing the changes right away without waiting for server confirmation.

Here are 2 ways a web browser could track changes:

- Local Storage: It's a simple key-value store, which stores the data persistently even after closing the browser. Yet it blocks the browser's main thread because of synchronous operations.
- IndexedDB: It's a NoSQL database and stores data persistently even after closing the browser. But it doesn't block the main thread because of its *asynchronous nature*.

A browser's main thread handles JavaScript execution, user interactions, and UI rendering. It means blocking the main thread will freeze the browser and affect user experience.

So Google Docs uses IndexedDB to handle offline changes.



IndexedDB Tracking Changes on a Web Browser

Google Docs runs 3 instances of IndexedDB on the browser:

- Operations store: It saves offline changes in a queue with correct ordering.
- Metadata store: It tracks timestamps and synchronization status with the remote server.
- Document store: It saves snapshots for document versioning.

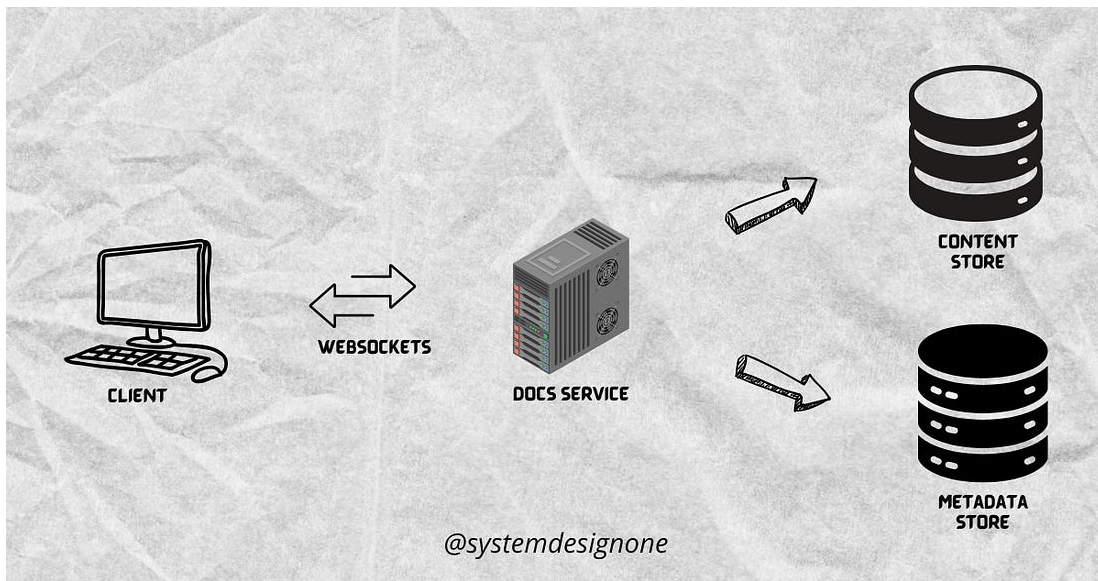
Besides IndexedDB removes older data after server acknowledgments periodically. Thus reducing the browser's storage needs.

Ready for the best part?

2. Conflict Resolution

Google Docs uses WebSockets for real-time bidirectional communication.

It maintains a persistent connection, which makes it easier to synchronize different parts of a document without conflicts.



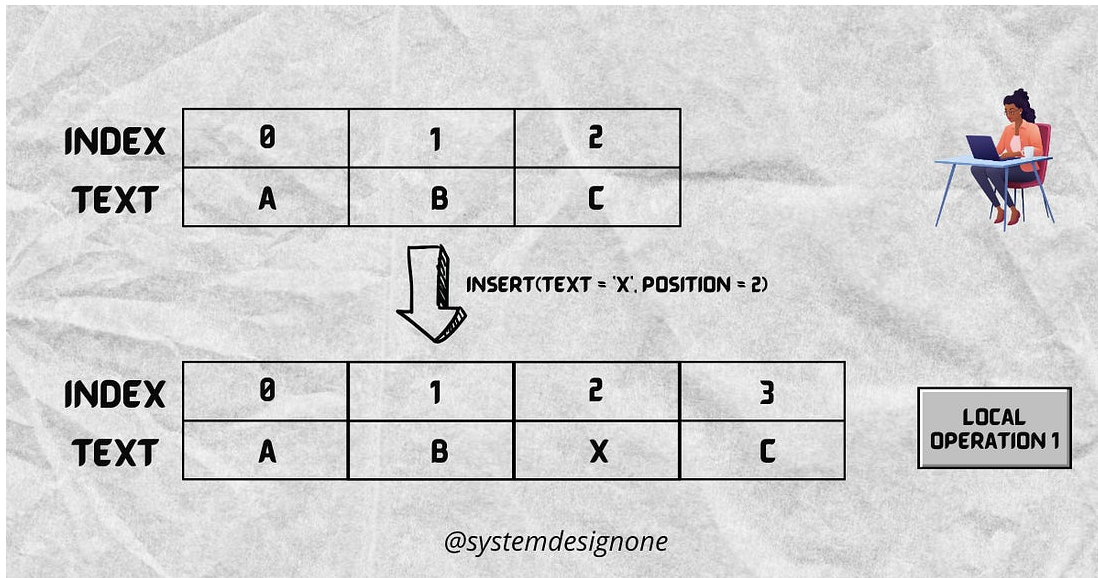
High-Level Architecture of Google Docs

They store document and media content durably in Google Colossus, a distributed file system. Imagine the **distributed file system** as a file system where data is spread across servers in different locations.

And store the metadata of documents in Google Spanner, a relational database.

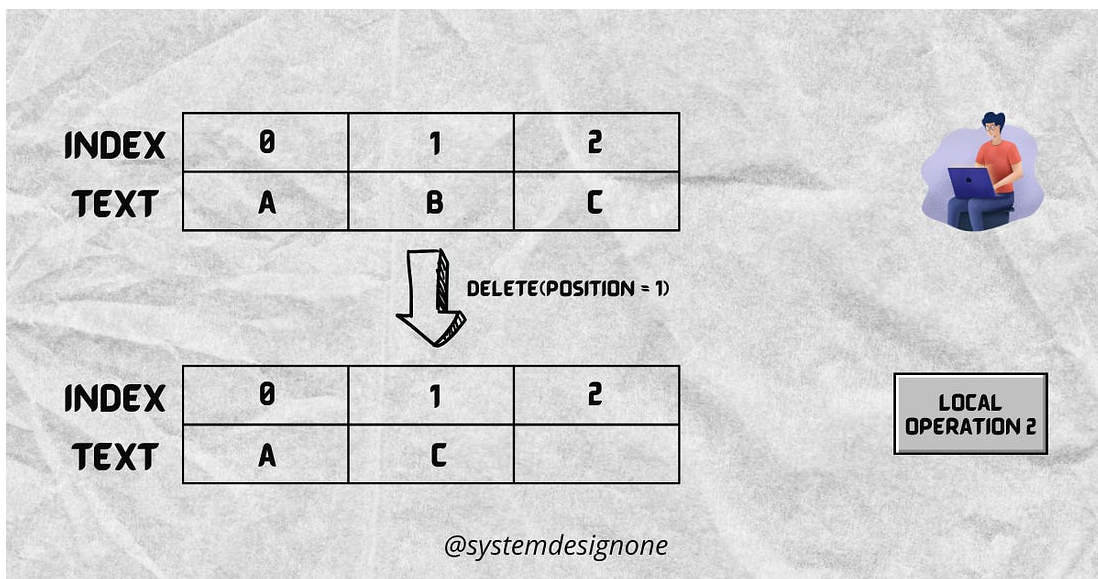
Here's the **Google Docs workflow**:

Imagine 2 people, Maria and George, working on the same Google Document.



Inserting Text Locally

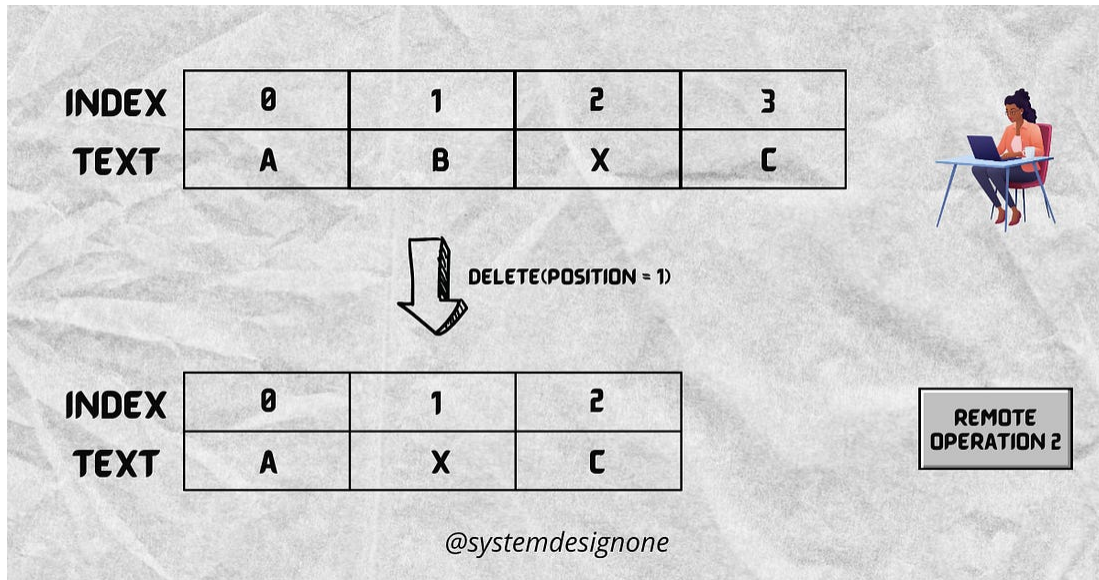
- Their local document copies contain the text 'ABC'.
- Maria inserts a text at position 2.
- So her local document shows the text 'ABXC'.



Deleting Text Locally

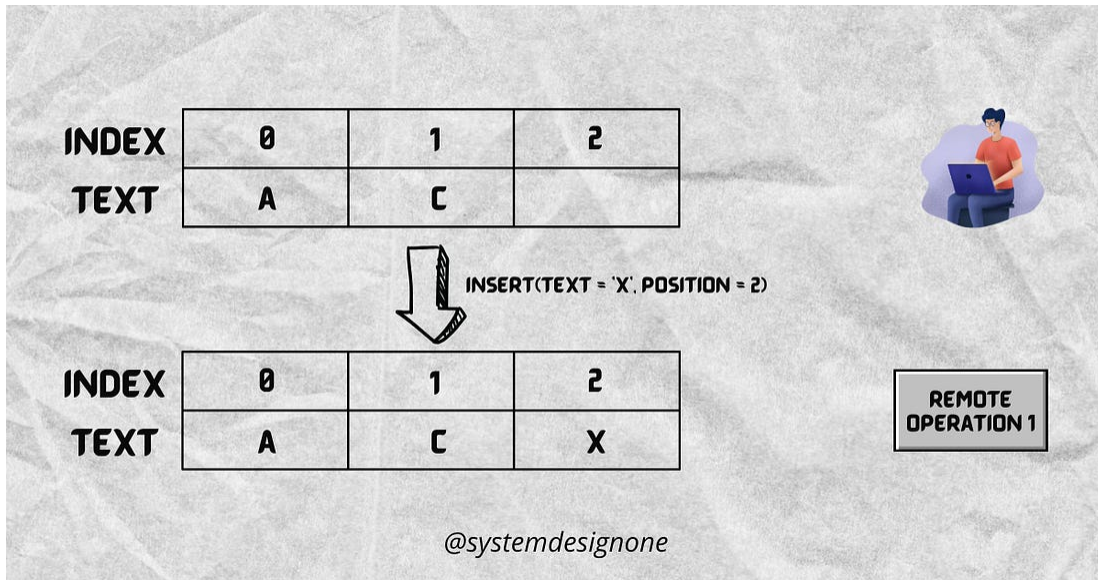
- While George deletes text in position 1 at the same time.

- So his local document shows the text 'AC'.



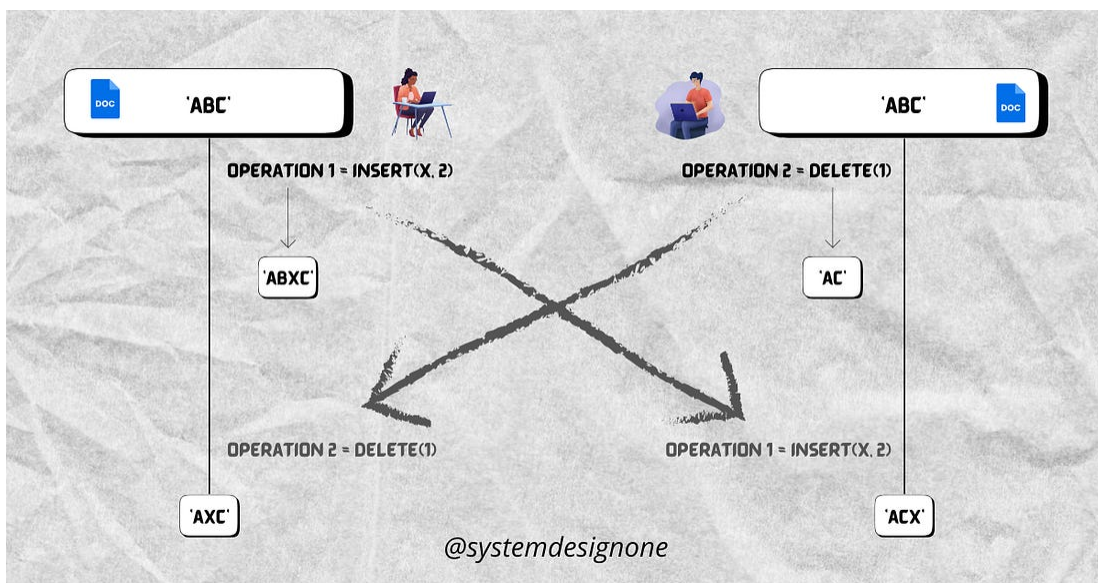
Applying a Remote Operation Locally

- Maria receives the remote operation from George, which deletes text.
- Her browser applies the remote operation locally to converge the document to the same state.
- So her local document copy shows the text 'AXC'.



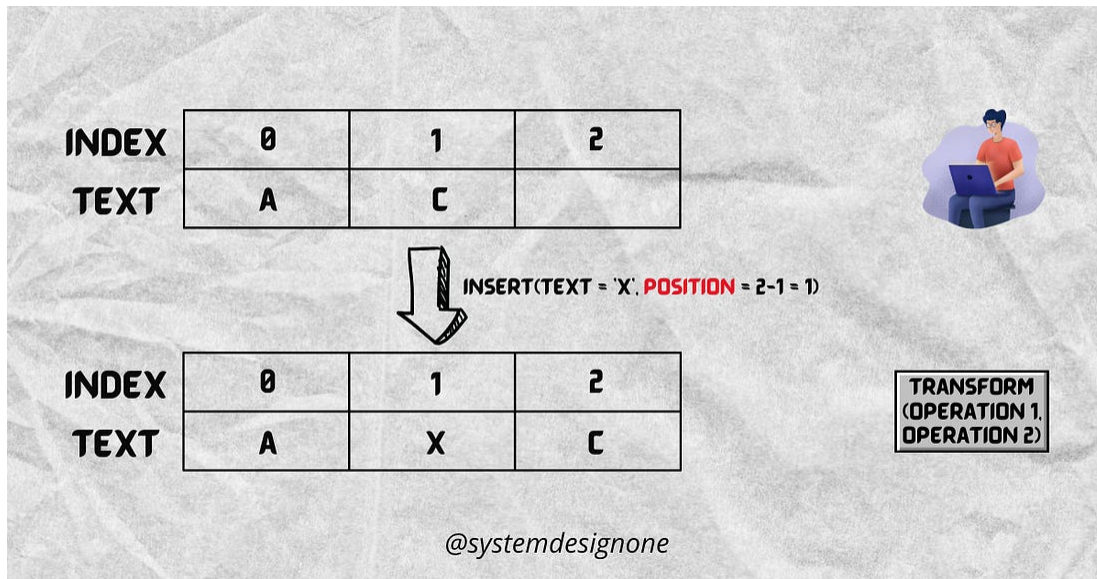
Applying a Remote Operation Locally

- While George receives the remote operation from Maria, which inserts text.
- His browser applies remote operation locally to converge the document to the same state.
- But his local document shows the text 'ACX'.
- So their local documents become out of synchronization.



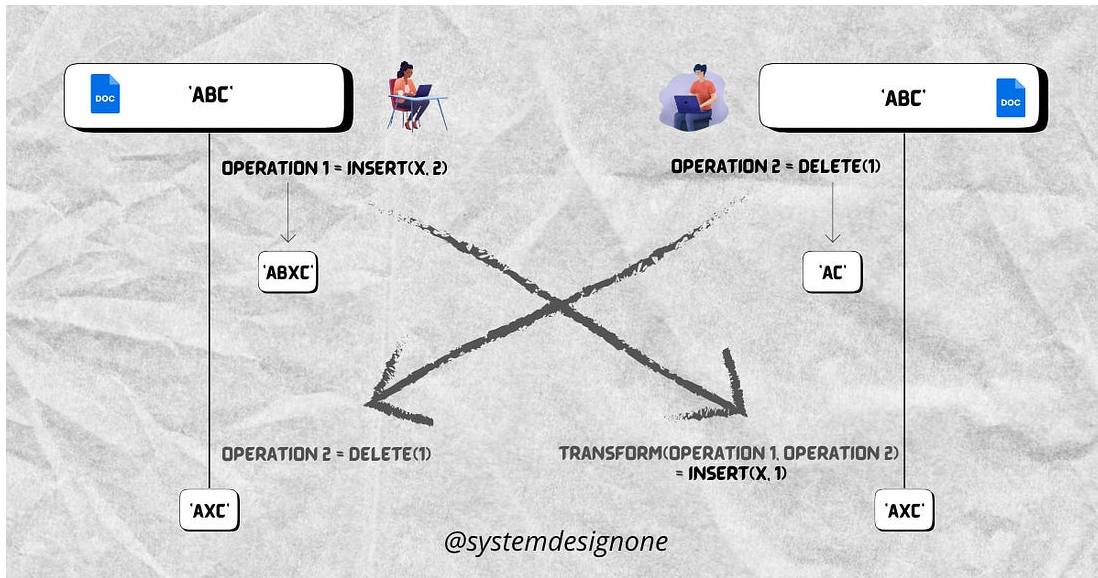
Local Document Copies With Inconsistent State

- The bottom line: applying a remote **concurrent operation locally without transformation** creates inconsistency.
- It means document copies won't converge to the same correct state.



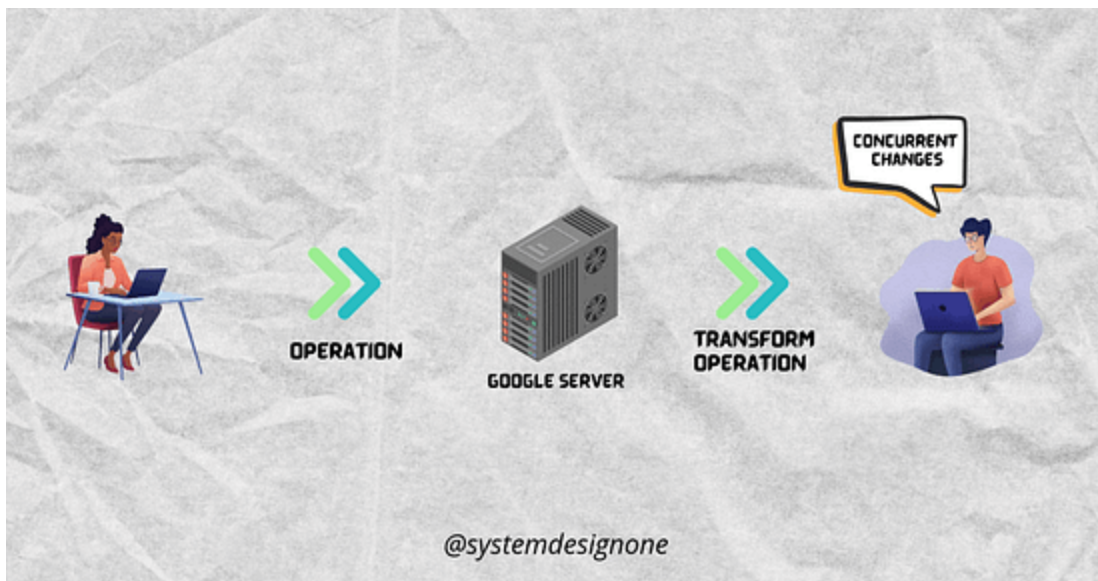
Shifting the Insert Operation's Position Backward Before Applying It

- So Google Docs transforms a remote concurrent operation by shifting its position.
- For example, the delete operation's position is before the insert operation's position.
- So George's browser shifts the insert operation's position backward, which is the remote operation, before applying it locally.
- Thus converging document copies to the same state.



Local Document Copies With Consistent State

- Put simply, **operational transformation shifts the position of concurrent operations to avoid conflicts if necessary.**



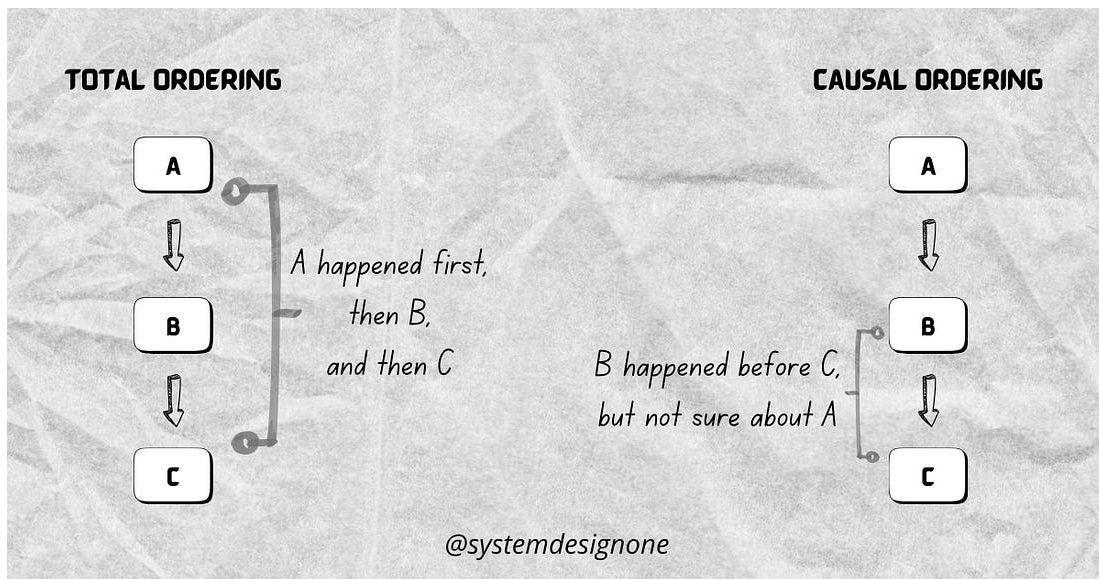
Operational Transformation Overview

Here's how operational transformation **resolves a conflict**:

1. The client applies an operation locally without delay.

2. The client sends the operation to the server via websockets.
3. The server broadcasts the operation to all clients asynchronously.
4. The receiving client applies OT locally and *transforms* the incoming operations if they're concurrent.
5. The server also applies OT in a race condition to resolve conflicts if necessary.

A combination of client and server-side OT makes Google Docs responsive and reliable.



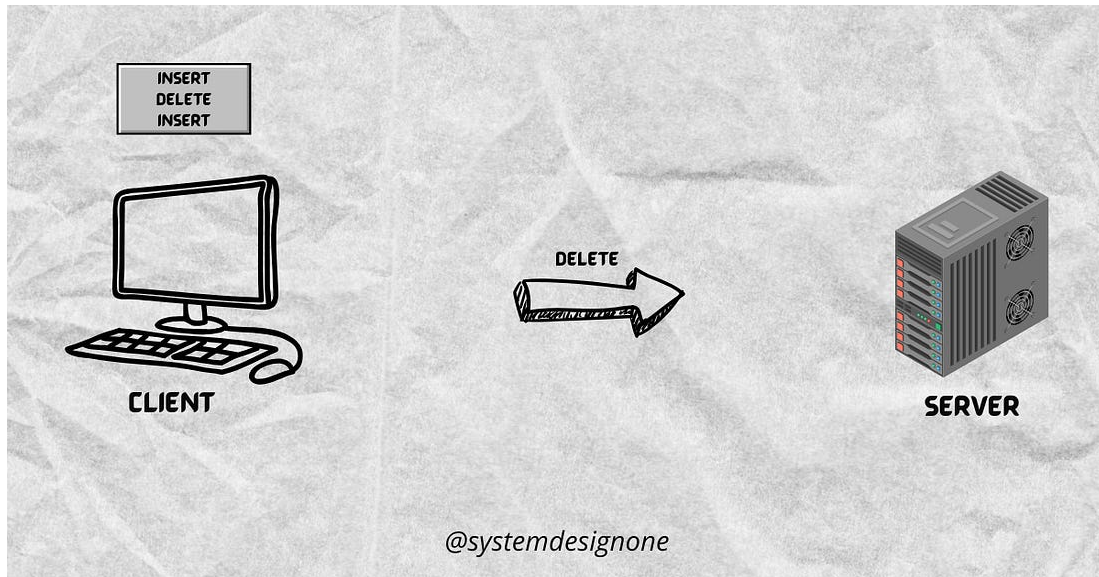
Operational Transformation Use Causal Ordering

Yet operational transformation doesn't support total ordering; instead, causal ordering.

Imagine **total ordering** as all events having a single agreed order, while **causal ordering** means events following a cause-and-effect relationship ($B \rightarrow C$).

The ordering of operations doesn't matter much because OT ensures consistency.

For example, the system knows if event B occurred before event C in causal ordering. It relies on logical clocks, such as the Lamport clock or Vector clock, for causal relationships. But event A is independent, so its order doesn't matter.

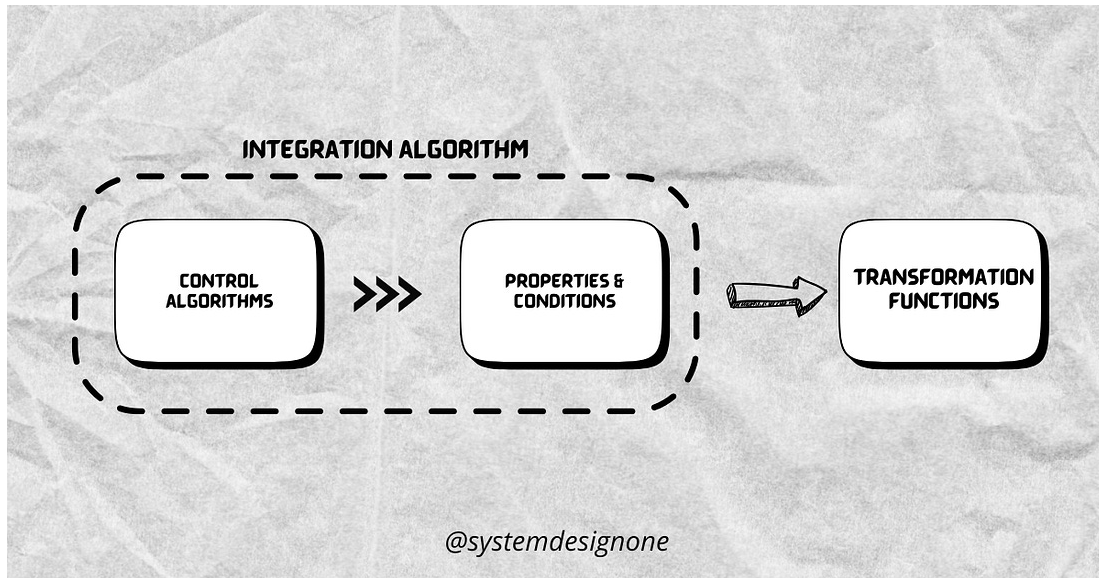


Composing Operations Before Sending Them to the Server

Also the client sends only a single operation to the server at a time.

Yet a user might do more operations before the server acknowledges the last operation. So the client stores extra operations in a buffer and composes them before sending them to the server.

The technique of combining operations is called **composition**. Besides server arranges operations in the correct order before broadcasting them to the client.



Operational Transformation Framework

Think of the **operational transformation framework** as a way for different people to edit the same document at once without conflicts. It has 2 parts:

- Integration algorithm: It receives operations and determines which transformation functions to run based on concurrency and content relations.
- Transformation function: It transforms operations based on its position and type.

Ready for the next technique?

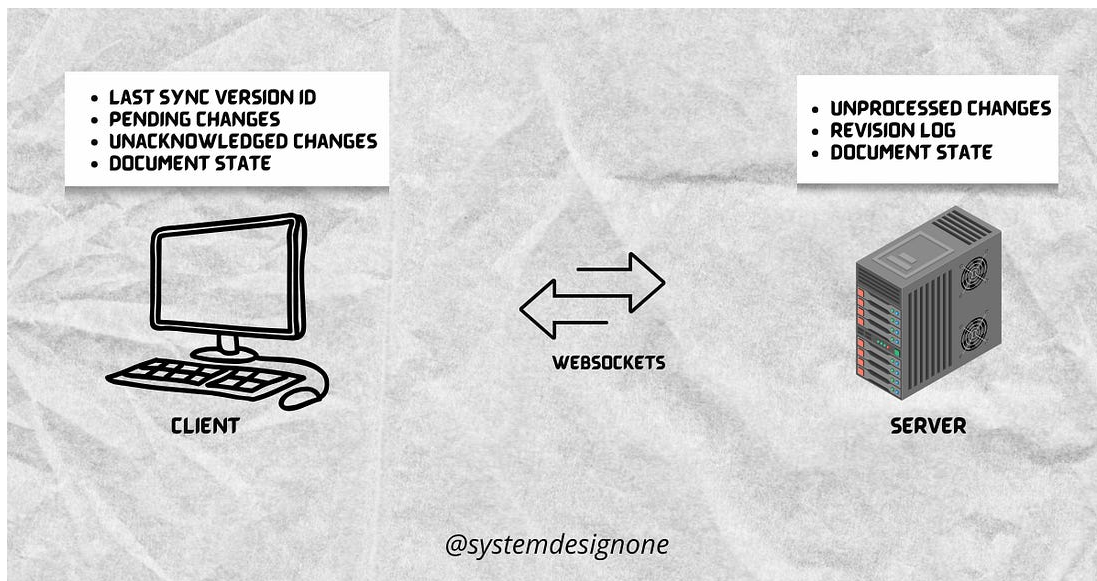
3. Change Synchronization

Google Docs relies on metadata for collaborative editing. Here's why:

- It's fast because it avoids network dependency by applying changes locally.
- It's accurate because each client has enough information to merge changes in a deterministic way.

- It's efficient because it sends only the minimum information over the network.

Also collaboration complexity remains constant because the server doesn't have to track each client's state.



Metadata to Synchronize Changes

The client keeps this metadata:

- Last sync version ID: It tracks the last document version synchronized with the server.
- Pending changes: It tracks local changes that weren't sent to the server yet.
- Unacknowledged changes: It tracks changes that weren't confirmed by the server yet.
- Document state: It tracks the local document state.

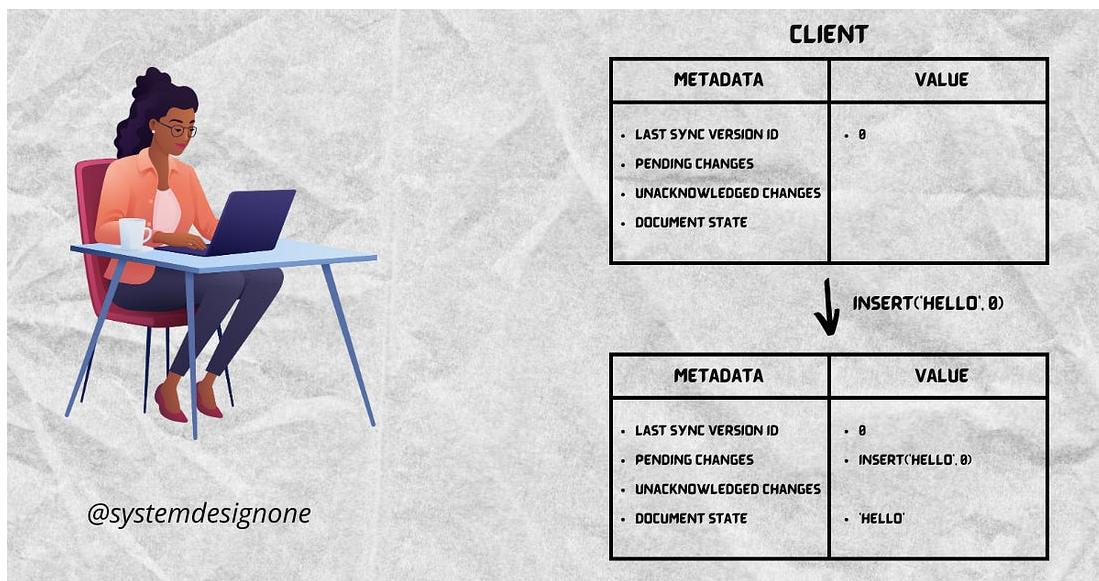
While the server stores this metadata:

- Unprocessed changes: It tracks changes that the server hasn't processed yet.

- Revision log: It tracks the list of processed changes.
- Document state: It tracks the server's state after applying the last change.

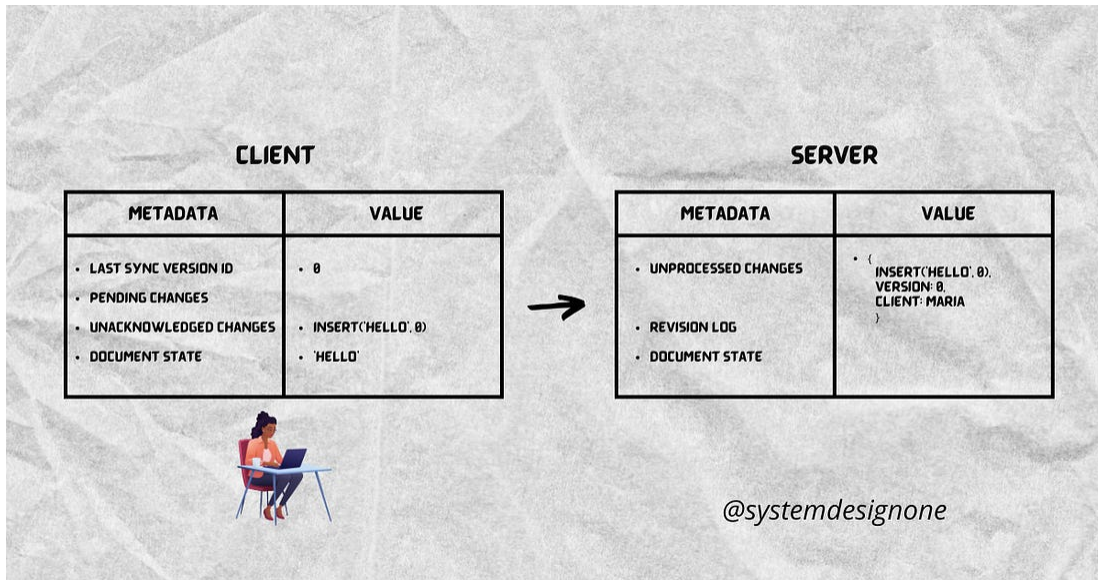
Here's the **synchronization workflow** of Google Docs:

Imagine 2 people, Maria and George, working on the same Google Document.



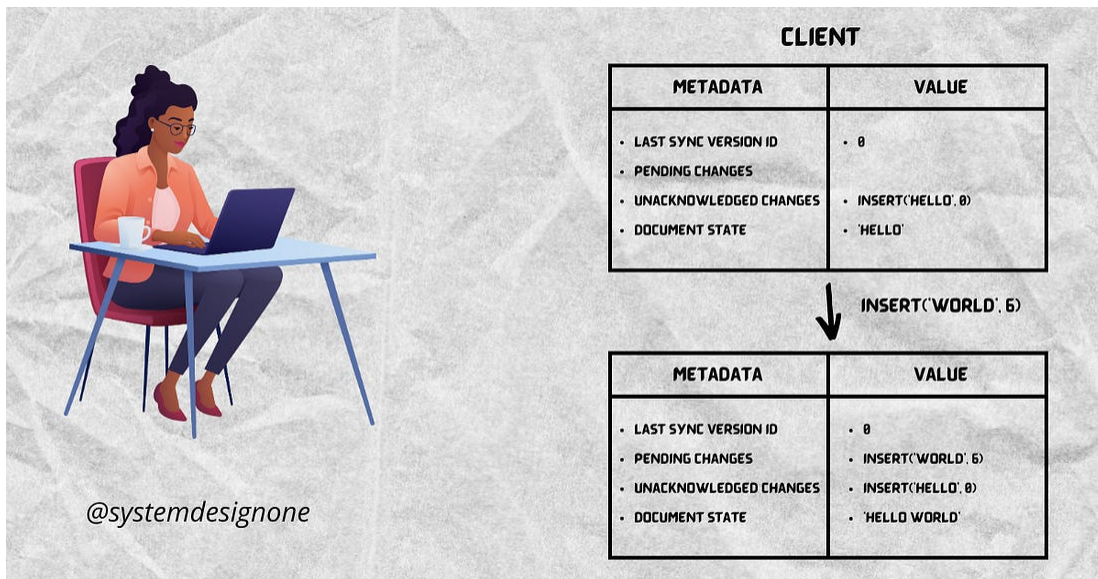
Adding Unsynchronized Changes to Pending Changes List

- Maria types "Hello" and her local document displays it right away.
- Her browser adds the text to its list of pending changes.



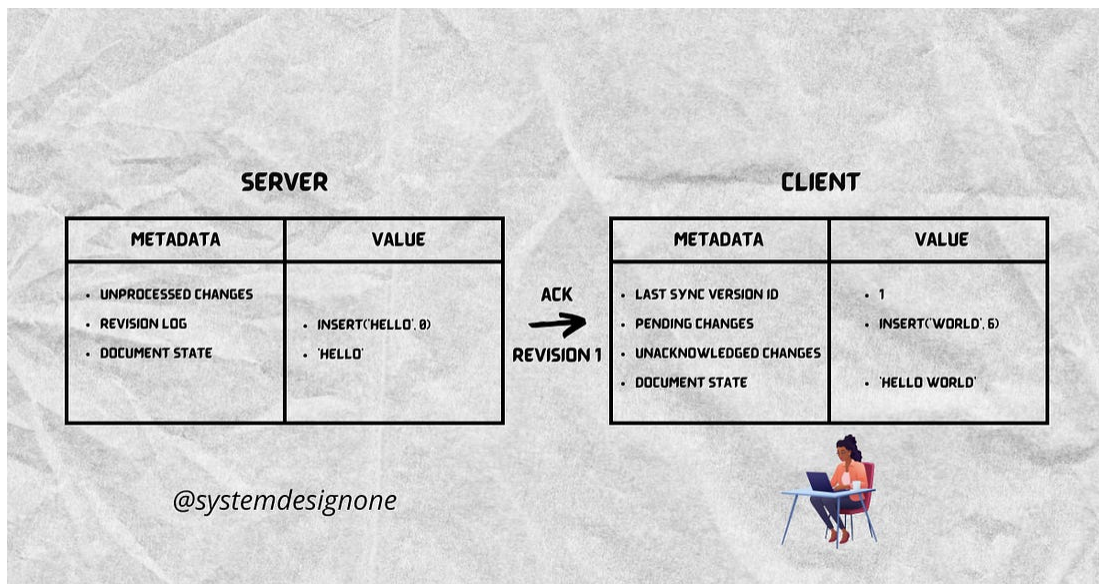
Sending Unsynchronized Changes to the Server

- The browser then sends her change to the server and adds it to its list of unacknowledged changes.
- The server saves her change in its list of unprocessed changes as it hasn't processed the change yet.



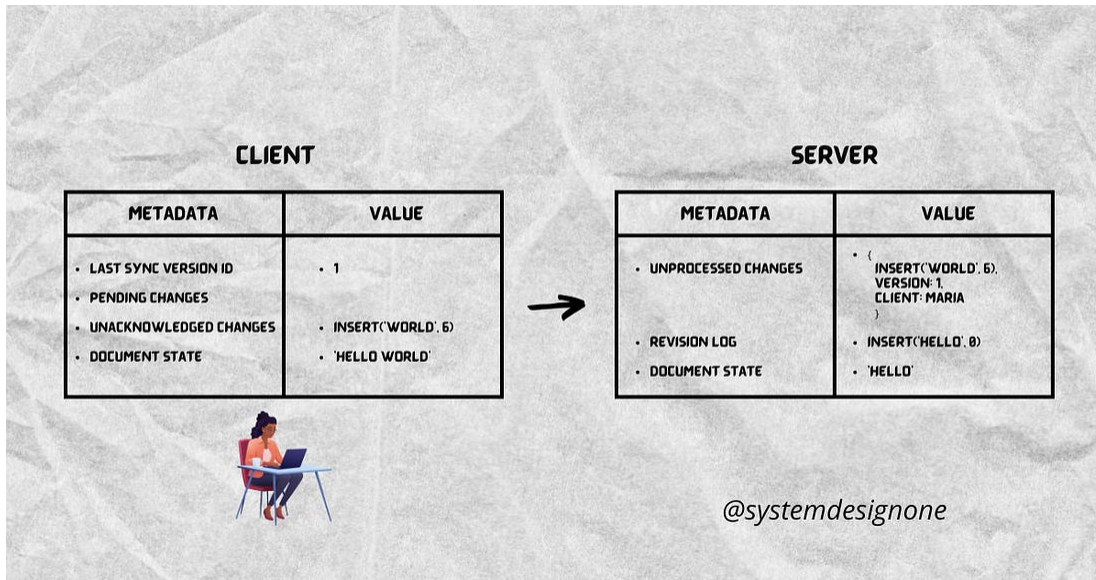
Adding a New Change to the Pending Changes List While Waiting for Acknowledgement

- Then she types “World” and the local document displays it right away.
- Yet the browser sends the newest change to the server only after it receives an acknowledgment for the last change.
- So the browser adds the newest change to its list of pending changes and doesn’t send it yet.



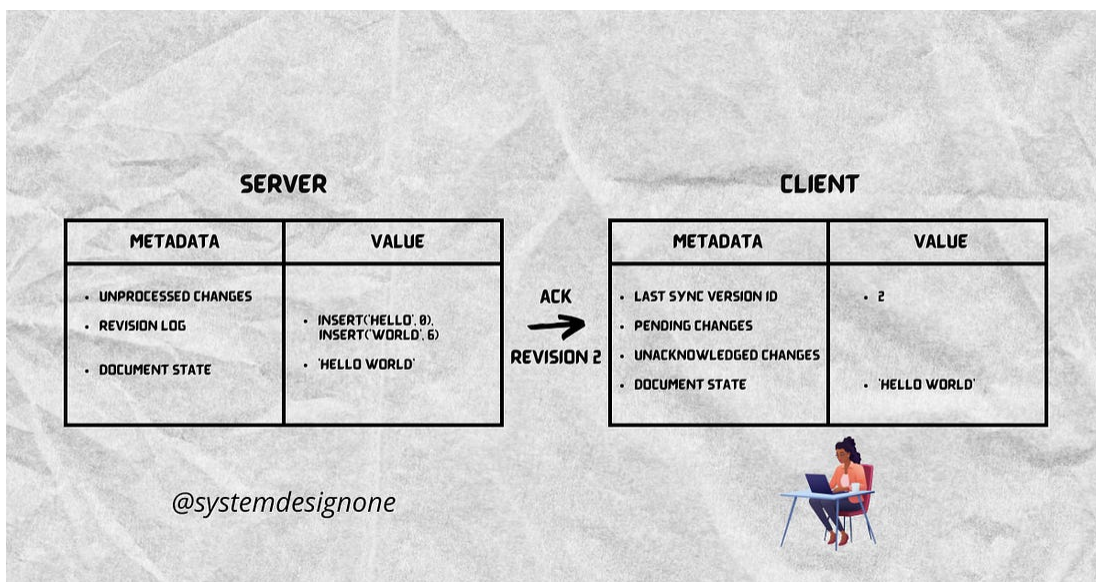
Recording Changes on the Revision Log

- In the meantime, the server processes her first change and adds it to the revision log.
- The server then sends an acknowledgment signal to the browser.
- So the browser updates its last sync version ID to 1.
- The browser then removes the first change from its list of unacknowledged changes.



Sending Newest Change to the Server

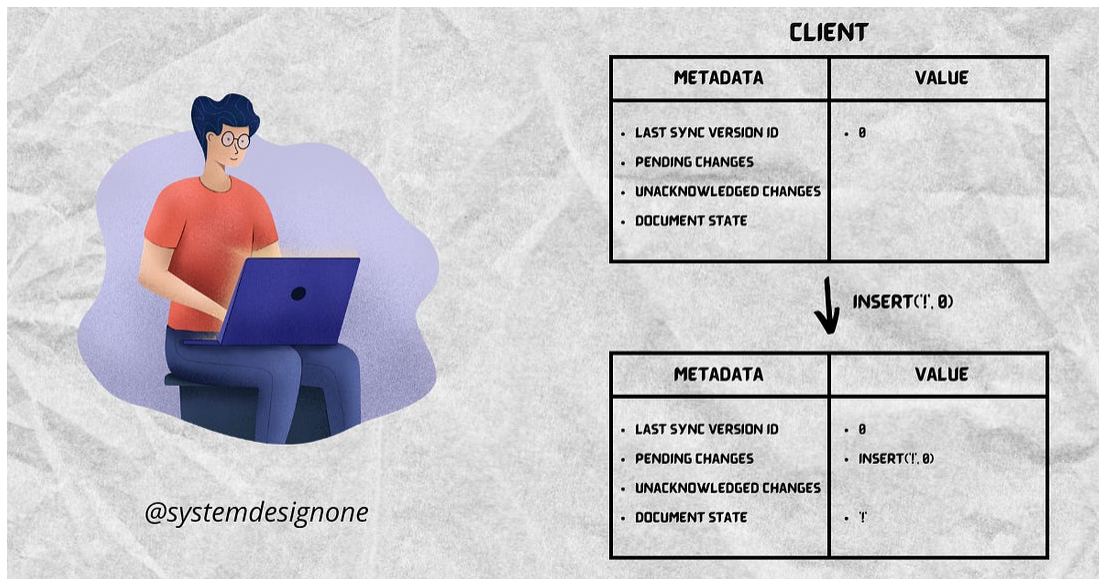
- The browser sends its newest change to the server.
- And adds the newest change to its list of unacknowledged changes.



Incrementing the Sync Version

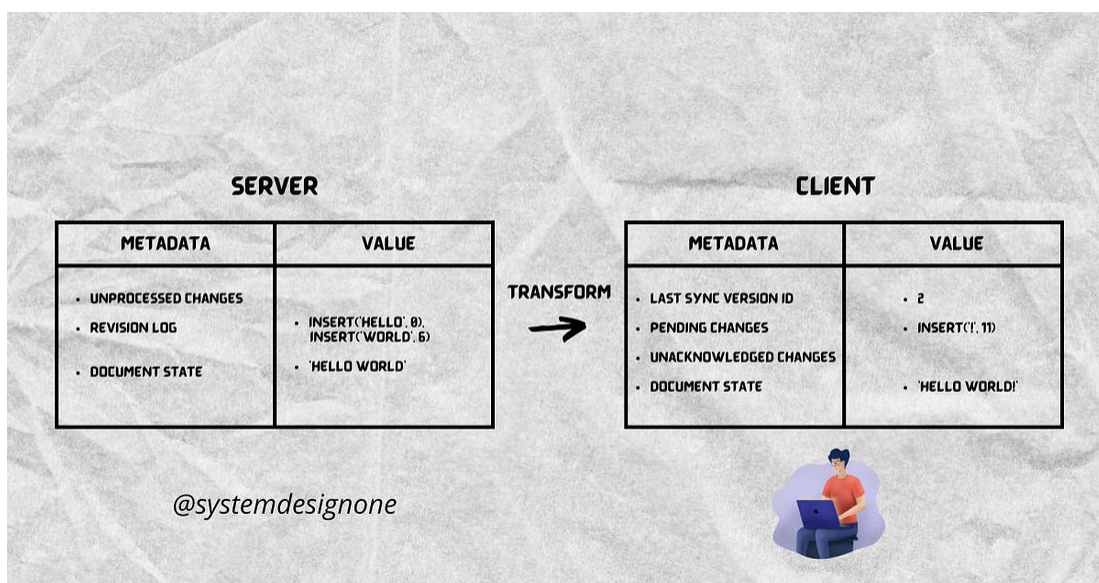
- While the server updates its revision log and sends an acknowledgment signal to her browser.

- So the browser updates its last sync version ID to 2.



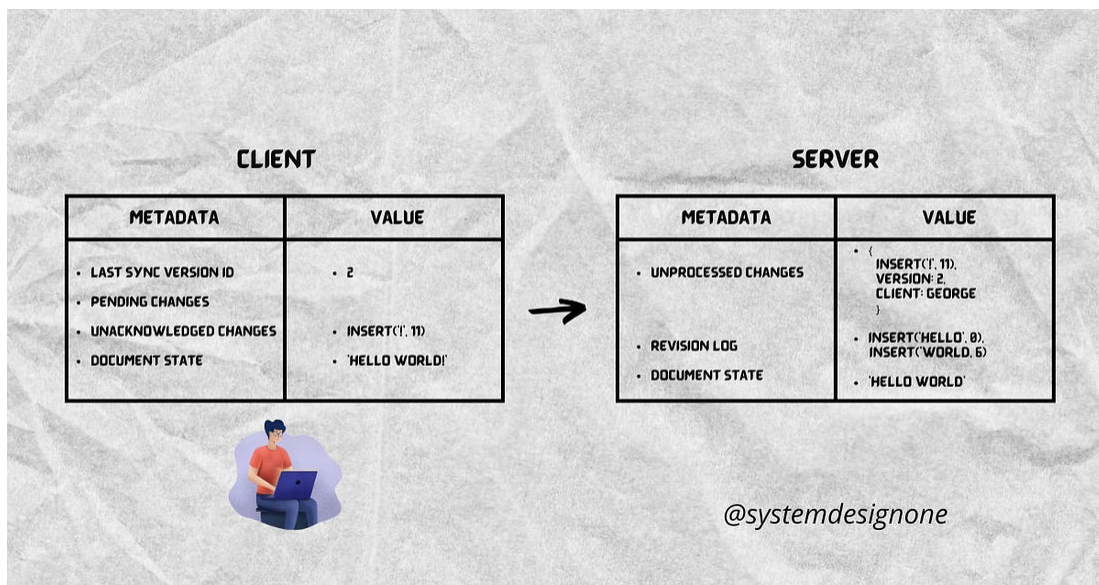
Adding a New Change to the Pending Changes List

- While George has a local copy of the same document, it's empty as he was offline.
- He types "!" and the local document displays it right away.
- His browser then adds the change to its list of pending changes.



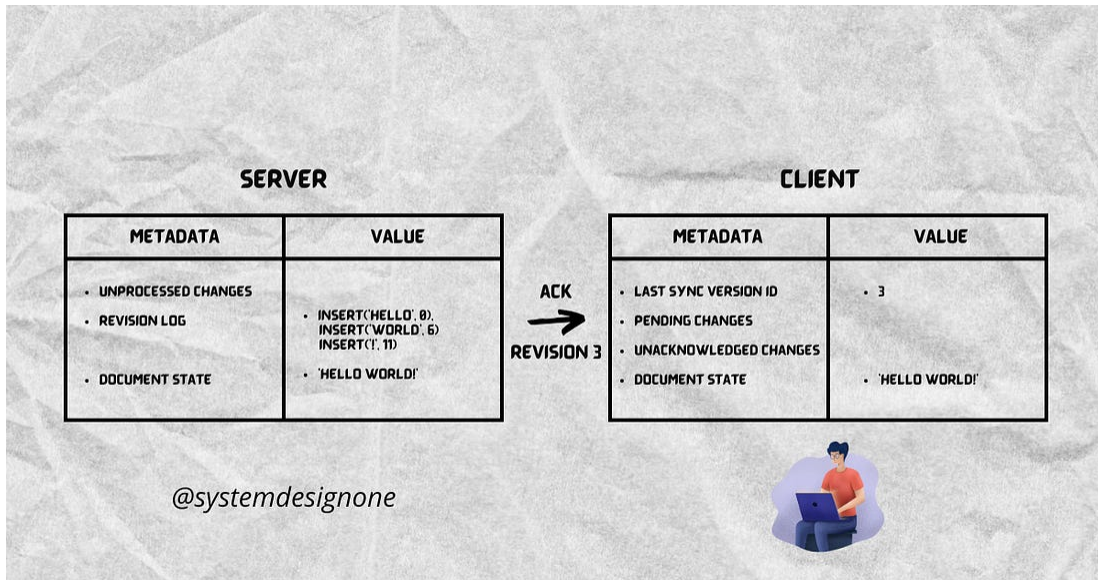
Fetching Changes From the Server

- His browser reconnects to the internet and receives the latest document version from the server.
- The browser transforms the incoming changes using operational transformation to avoid conflicts.
- And shifts the local insert operation's position from 0 to 11, then reapplies the change.



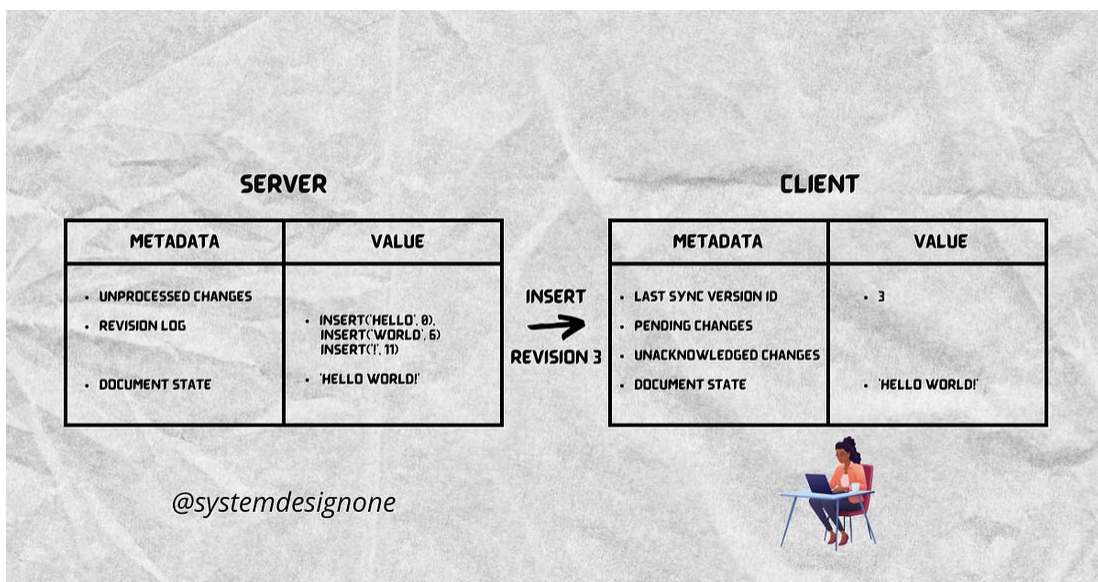
Sending Transformed Changes to the Server

- His browser sends the transformed change to the server.
- The browser then adds the change to its list of unacknowledged changes.



Server Sending an Acknowledgement Signal

- The server updates its revision log and sends an acknowledgment signal to the browser.
- So his browser updates its last sync version ID to 3.



Sending the Newest Change to Others

- The server then sends the change to Maria as well.

Thus both of them have a synchronized document.

Let's keep going!

Google Docs represents each **text style** as an operation.

For example, the operation to underline a text is:

```
{style underline "text range"}
```

They transform a style using operational transformation only if there's a *concurrent text change*.

Yet 2 *concurrent style changes* to the same text don't conflict with each other if they're for different style attributes. For example, a change in text color and underlining the same text at the same time don't conflict with each other.

And they use the last write wins mechanism if there are concurrent changes to the same style attribute. For example, concurrent changes to the color of a text.

So transformation is unnecessary for concurrent style changes.

INDEX	0	1	2	3
STYLE(UNDERLINE, 1-2)	A	<u>B</u>	<u>C</u>	D
STYLE(COLOR:RED, 2-3)	A	B	C	D
	A	<u>B</u>	<u>C</u>	D

@systemdesignone

Concurrent Style Changes to the Same Text

For example, here are 2 concurrent style changes:

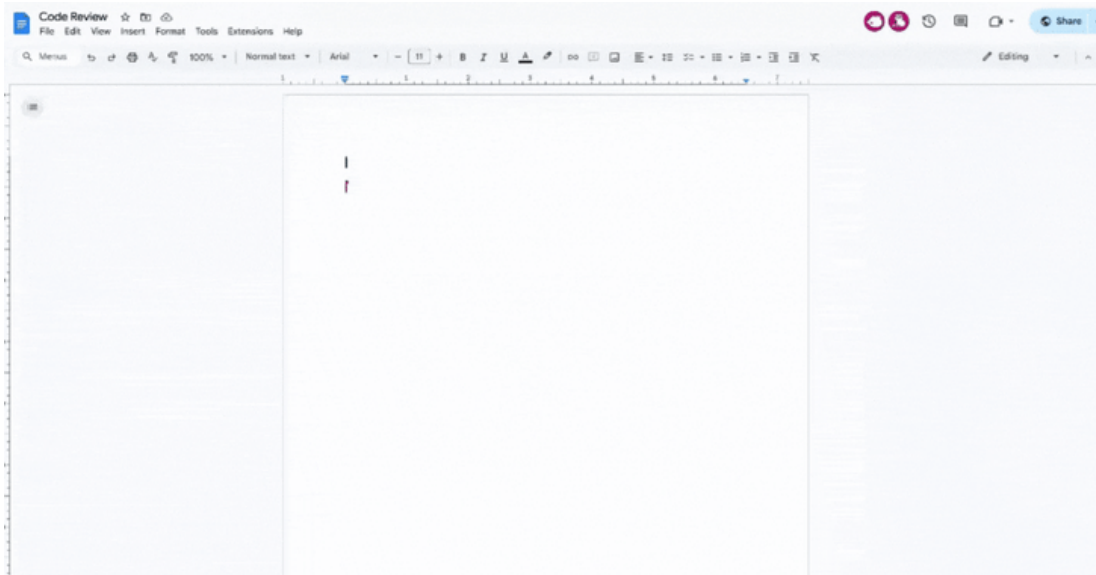
- { style underline "1-2" }
- { style font-color: red "2-3" }

The text in the range "1-2" can be underlined and also in red. So a transformation is unnecessary.

They represent a user's cursor on a shared document with a unique color.

Google Docs relies on websockets and operational transformation to synchronize cursor position across users in real-time.

Also a concurrent operation on the same text will shift a user's cursor position.

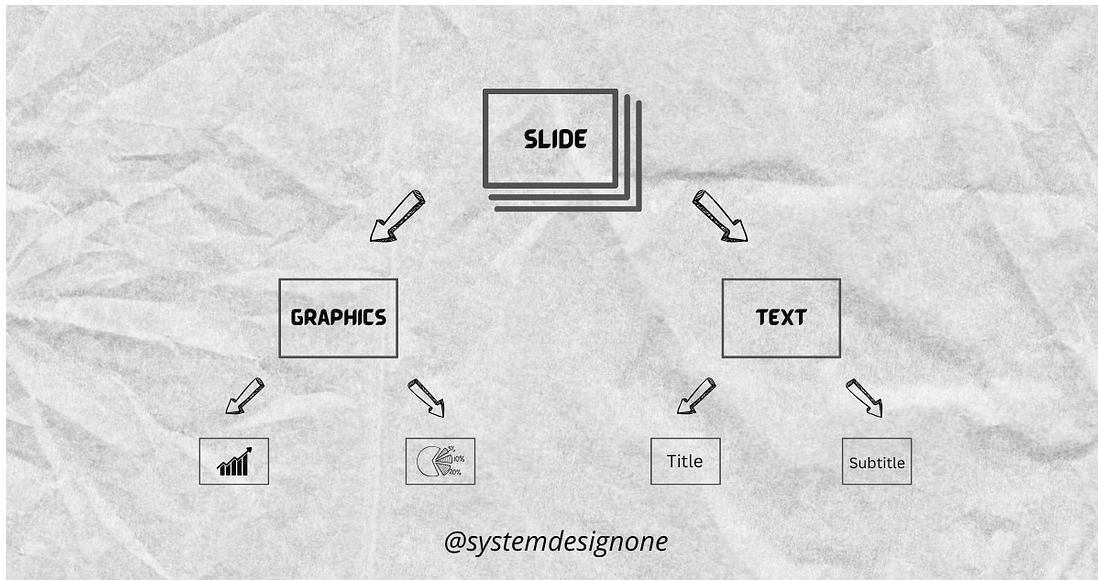


Cursor Position in Real-Time Collaborative Editing

Here's how they synchronize the **cursor position**:

- They use browser DOM events to track the cursor position.
- The client then sends the cursor's position through websockets.
- The server broadcasts the cursor position to other clients.
- The cursor position shifts forward if the text is inserted before the cursor.
- While the removal of text before the cursor shifts its position backward.

Besides an offline user sends their cursor's last known position to the server when they reconnect. The server then shares it with others. Thus there's low performance overhead.



Operational Transformation in Google Slides

Google Slides also uses operational transformation (**OT**).

OT assumes data objects to be linearly addressable. Yet data objects don't have to be in text form; instead, they can be graphic objects or slides.

Put simply, slides extends the linear address space into a hierarchical data model.

They arrange slides in a linear order. Each slide contains text and graphics, which are represented hierarchically. And OT handles text inside a text box linearly.



👋 Find me on [LinkedIn](#) | [Twitter](#) | [Threads](#) | [Instagram](#)

Thank you for supporting this newsletter. Consider sharing this post with your friends and get rewards. Y'all are the best.

1 Referral



**Leetcode
Master
Template**

2 Referrals



**Interview
Mistakes
to Avoid PDF**

3 Referrals



**Popular
Interview
Questions PDF**

Share

TL;DR 🕒

You can find a summary of this article [here](#). Consider reposting if you find it helpful.

References

- [Operational Transformation in Real-Time Group Editors: Issues, Algorithms, and Achievements](#)
- [Tombstone Transformation Functions for Ensuring Consistency in Collaborative Editing Systems](#)
- [What's different about the new Google Docs: Working together, even apart](#)
- [What's different about the new Google Docs: Conflict resolution](#)
- [What's different about the new Google Docs: Making collaboration fast](#)
- [Operational Transformation or How Google Docs Works](#)
- [High-Latency, Low-Bandwidth Windowing in the Jupiter Collaboration System](#)
- [Issues and Experiences in Designing Real-time Collaborative Editing Systems](#)
- [Distributed Systems Operational Transformation](#)
- [Federation Protocol](#)
- [Differential Synchronization Whitepaper](#)
- [Differential Synchronization Google TechTalks](#)